

Trapdoor

Architecture Flow

A scanner for prompt-injection payloads hidden in user-uploaded files — PDFs, images, DOCX, HTML, video. Extracts every fragment of text (including invisible PDF layers, OCR, metadata, HTML comments, and LSB-decoded image data), runs six detectors in parallel, sanitizes what survives, and returns a verdict plus the LLM-safe context.

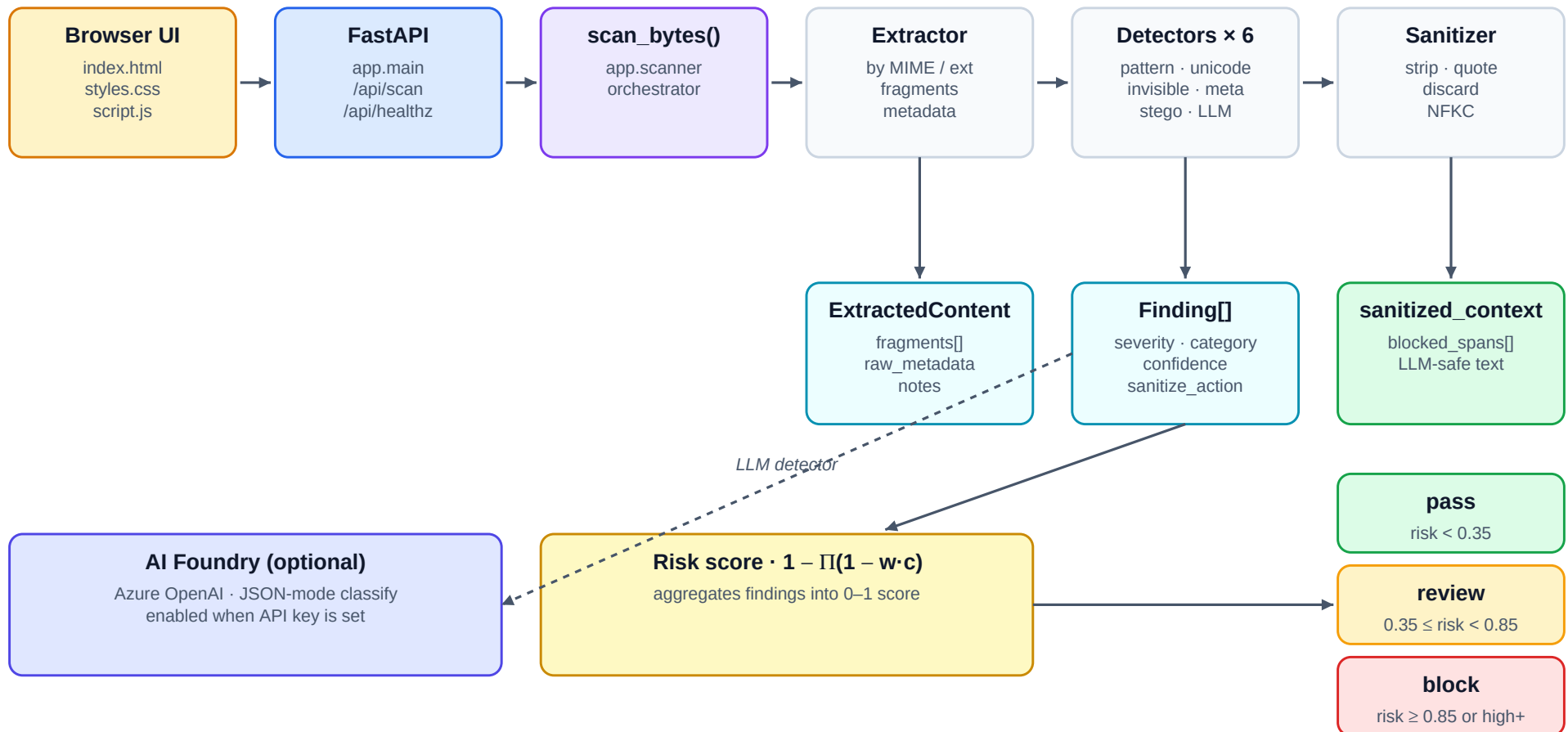
Pipeline at a glance

1	Upload	Browser POSTs multipart/JSON to FastAPI.
2	Extract	MIME-routed extractor pulls fragments + metadata.
3	Detect	Six detectors emit Findings with severity / confidence.
4	Sanitize	Block, quote, or strip each fragment based on its action.
5	Score	Probabilistic aggregation → pass / review / block.
6	Classify (opt.)	Azure AI Foundry confirms suspicious fragments.

See the next page for the full data-flow diagram, followed by component-level detail (extractors, detectors, sanitizer, scoring).

Scan Pipeline

Browser → FastAPI → Extract → Detect → Sanitize → Score → (optional) AI Foundry



Solid arrows = always run. Dashed = optional path (Azure OpenAI key required).

Components

Extractors — backend/app/extractors/

Extractor	Pulls	Notes
PDFExtractor	Per-character text + color; marks invisible (≥ 0.95 RGB) fragments separately. Metadata.	pdfplumber. CMYK approximated to RGB.
ImageExtractor	OCR · LSB-plane entropy + decoded ASCII · EXIF · histogram heuristic.	pytesseract optional.
DocxExtractor	Body paragraphs + core properties (author / title / subject / comments / keywords).	python-docx.
TextExtractor	Body + every HTML comment as kind="comment" fragment.	UTF-8 with latin-1 fallback.
VideoExtractor	Frame-by-frame OCR at 1 fps.	cv2 + pytesseract optional.

Detectors — backend/app/detectors/

Detector	Strategy
PatternDetector	Curated regex set (instruction-override, role-spoofing, exfiltration, jailbreak, guardrail-removal, score-manipulation). Bumps confidence when the fragment is invisible, metadata, or a comment.
UnicodeDetector	Counts ZWSP / ZWNJ / ZWJ / WJ / BOM. Detects Latin-dominant strings with foreign-script characters (confusables).
InvisibleTextDetector	Flags any fragment that extractors tagged visibility=invisible.
MetadataDetector	Instruction-shaped text inside metadata fields.
SteganographyDetector	Inspects kind=decoded fragments — high severity when decoded text contains exfiltration targets.
LLMDetector	Calls Azure AI Foundry to classify each substantial fragment. Skipped when no API key is set.

Sanitizer — backend/app/sanitizer.py

Per fragment: **visibility=invisible** → blocked. **kind=comment** → blocked. **kind=metadata** → reframed as [metadata.x] Findings requesting **discard** → blocked. Findings requesting **quote** (or kind=ocr/decoded) → wrapped in an untrusted fenced block. All surviving text is NFKC-normalized with zero-width chars stripped. The result is **sanitized_context** — exactly what downstream code passes to the LLM.

Risk score — backend/app/scanner.py

Each finding is treated as an independent probability that the file is malicious. The scanner aggregates $1 - \Pi(1 - severity_weight \times confidence)$ into a 0–1 risk score:

block	risk ≥ 0.85, or any high/critical finding
review	$0.35 \leq risk < 0.85$
pass	risk < 0.35

Extension points

- New modality:** drop a class in backend/app/extractors/, register it in registry._BY_EXT.
- New attack pattern:** append a tuple to pattern_detector._PATTERNS.
- New detector:** subclass Detector, return Finding[], add it to detectors.all_detectors().
- New sanitize policy:** edit sanitizer.sanitize() — every detector already declares the action it wants.